

Comunicação confiável e comunicação orientada a mensagens

STD29006 – Engenharia de Telecomunicações

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Comunicação confiável

Comunicação confiável I

- **Comunicação confiável** garante que as mensagens enviadas por um processo sejam recebidas corretamente pelo processo de destino, mesmo diante de falhas
 - **Perda de mensagens**
 - Mensagens podem ser perdidas durante a transmissão
 - **Duplicação de mensagens**
 - Mensagens podem ser recebidas mais de uma vez
 - **Entrega fora de ordem**
 - Mensagens podem chegar em uma ordem diferente da que foram enviadas

Comunicação confiável II

- **Se usar um canal confiável**, então a própria camada de transporte (ex: TCP) garante a entrega correta das mensagens
- **Se usar um canal não confiável**, então a aplicação deve implementar mecanismos para garantir a entrega correta das mensagens

Tipos de falhas em canais e processos

■ Falhas em canais de comunicação

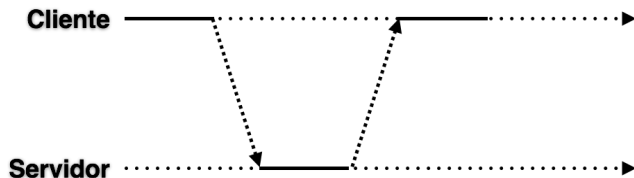
- Queda, omissão, *timeout*, arbitrária
- Canais confiáveis mascaram omissão/queda

■ Falhas em processos

- Omissão de envio/recepção
- Parada (crash)
- Arbitrária (comportamento incorreto)

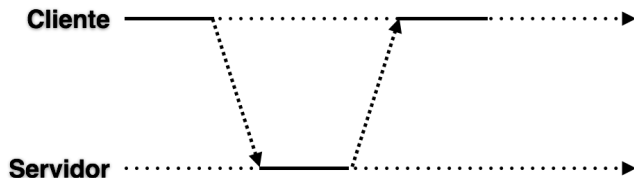
- TCP mascara falhas por omissão (uso de *ack* e retransmissões)
- TCP não mascara falhas por queda do processo, quando a conexão é interrompida abruptamente

Comunicação no modelo cliente-servidor



- 1 **Cliente** envia um pedido para o **servidor**
- 2 **Servidor** processa o pedido e envia uma resposta
- 3 O cliente aguarda a resposta do servidor

Comunicação no modelo cliente-servidor

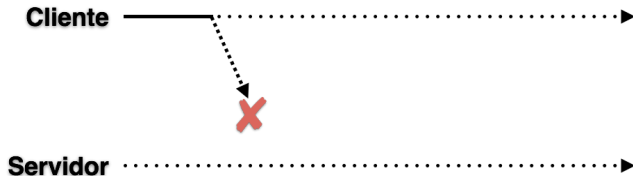


- 1 **Cliente** envia um pedido para o **servidor**
- 2 **Servidor** processa o pedido e envia uma resposta
- 3 O cliente aguarda a resposta do servidor

Quais são os problemas que podem ocorrer durante a comunicação?

Comunicação no modelo cliente-servidor

Servidor pode não receber o pedido

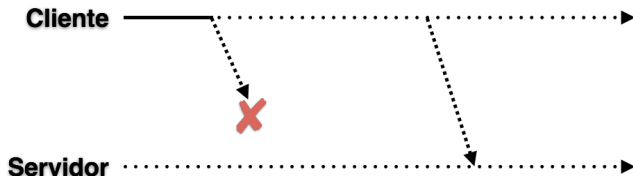


■ O que o cliente deve fazer?

- 1 Detectar que o pedido foi perdido por meio de um *timeout*
- 2 Reenviar o pedido

Comunicação no modelo cliente-servidor

Servidor pode não receber o pedido

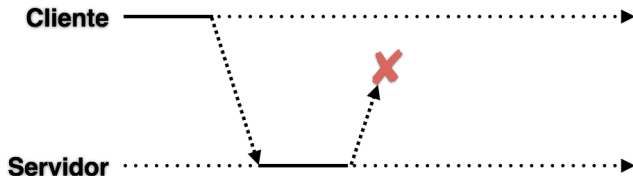


■ O que o cliente deve fazer?

- 1 Detectar que o pedido foi perdido por meio de um *timeout*
- 2 Reenviar o pedido

Comunicação no modelo cliente-servidor

Resposta pode não chegar ou servidor demorar para responder

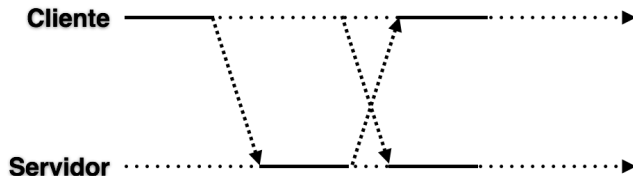


■ O que o cliente deve fazer?

- 1 Aguardar pelo tempo de expiração (*timeout*)
- 2 Enviar um novo pedido

Comunicação no modelo cliente-servidor

Resposta pode não chegar ou servidor demorar para responder

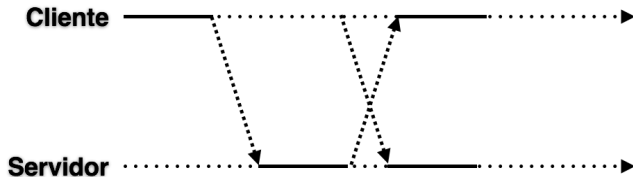


■ O que o cliente deve fazer?

- 1 Aguardar pelo tempo de expiração (*timeout*)
- 2 Enviar um novo pedido

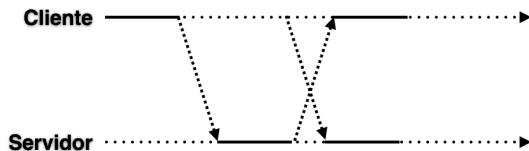
Comunicação no modelo cliente-servidor

Resposta pode não chegar ou servidor demorar para responder



- O servidor trata todo pedido como um novo pedido, ou seja, não sabe se já processou o pedido ou não
- Este comportamento só é apropriado para **requisições idempotentes**

Requisições Idempotentes



■ Requisições idempotente

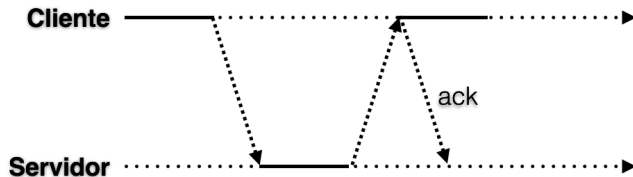
- Podem ser repetidas várias vezes e sempre produzirão o mesmo resultado, ou seja, o estado do servidor não muda
- Exemplo: HTTP GET obtém dados no servidor sem alterá-los

■ Requisições não idempotentes

- Podem alterar o estado do servidor e gerar resultados diferentes se executadas mais de uma vez
- Exemplo: HTTP POST cria um novo recurso no servidor, alterando seu estado

Comunicação no modelo cliente-servidor

Resposta pode não chegar ou servidor demorar para responder



■ Como lidar com requisições não idempotentes?

- Servidor mantém um histórico com todos os pedidos anteriores e exige que o cliente confirme a entrega (ACK)
 - cada pedido tem um identificador único

Semântica de entrega I

Como o cliente deve lidar com a entrega de mensagens

- **No máximo uma vez** (*at-most-once*)

- Se o cliente recebeu resposta, então o pedido foi processado uma única vez pelo servidor

- **Ao menos uma vez** (*at-least-once*)

- Se o cliente recebeu resposta, então o pedido foi processado no mínimo uma vez, mas pode ter sido processado mais vezes pelo servidor

- **Exatamente uma vez** (*exactly-once*)

- Se o cliente recebeu resposta, então o pedido foi processado exatamente uma vez pelo servidor

Em sistemas distribuídos, a semântica ***exactly-once*** é difícil de implementar, então implementações típicas usam ***at-most-once*** ou ***at-least-once***

Semântica de entrega II

Como o cliente deve lidar com a entrega de mensagens

- ***at-most-once*** pode ser implementado pelo servidor **com histórico**
 - Armazena o ID das mensagens recebidas e ignora mensagens duplicadas
 - Um pouco mais complexo para implementar, porém tolerante a faltas
- ***at-least-once*** pode ser implementado pelo servidor **sem histórico**
 - Reenvia mensagens até receber um ACK do cliente
 - Simples para implementar, mas não é tolerante a faltas
- ***exactly-once*** é o comportamento padrão de chamadas de procedimento local
 - O método é chamado e o resultado é retornado

Resumo da comunicação confiável

- TCP mascara falhas por omissão (uso de *ack* e retransmissões), mas não mascara falhas por queda do processo
- Sistemas distribuídos precisam lidar com
 - Reenvios
 - Duplicação de mensagens
 - Falhas silenciosas
- **Idempotência** é fundamental para garantir a entrega correta das mensagens

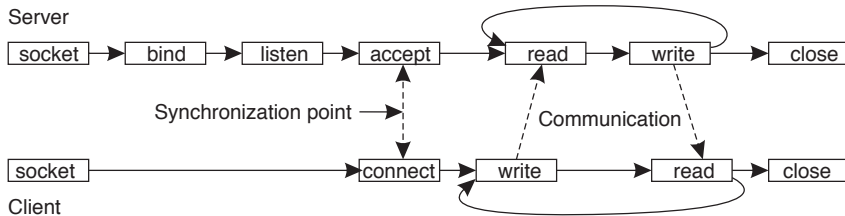
Comunicação orientada a mensagens

Revisão sobre modelo de comunicação cliente e servidor

Paradigma pedido e resposta

- **Endereçamento** – como localizar o servidor?
 - Conhecimento prévio, *broadcast* ou serviço de nomes
- Comunicação **síncrona** ou **assíncrona**
 - Síncrona – emissor/receptor fica bloqueado até enviar/receber mensagem
 - Assíncrona – emissor não fica bloqueado e o retorno também não bloqueia

Comunicação transitória



- RPC e RMI são exemplos de comunicação **transitória**
 - Emissor e receptor precisam estar ativos ao mesmo tempo
 - As mensagens são trocadas diretamente entre os processos emissor e receptor

Modelos de comunicação: *pull* vs *push*

■ Modelo *pull*

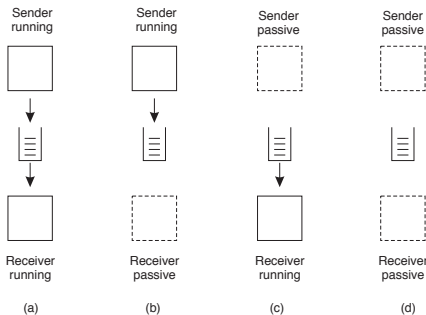
- Cliente solicita dados ao servidor (ex: HTTP)
- Servidor não precisa manter estado (*stateless*)
- Menor escalabilidade

■ Modelo *push*

- Servidor envia dados sem solicitação explícita (ex: streaming)
- Requer estado no servidor (*stateful*)
- Maior escalabilidade

Comunicação persistente por mensagens

- Em redes WAN, conexões entre sistemas podem ser instáveis
- Comunicação persistente garante entrega mesmo com falhas temporárias
- Comunicação ocorre por meio de filas de mensagens (assíncrona)
 - Mensagens ficam armazenadas até o receptor estar disponível



Middleware orientado a mensagens (MOM)

- **Facilita comunicação desacoplada no tempo entre aplicações**
 - Emissor e receptor não precisam estar ativos ao mesmo tempo
- **Garante persistência e roteamento das mensagens**
 - Mensagens podem ser roteadas por diversos servidores intermediários
- **Permite interoperabilidade entre sistemas heterogêneos**
 - Comunicação entre aplicações é intermediada por filas de mensagens

Interface básica para sistemas de filas de mensagens

■ PUT

- Colocar uma mensagem em uma fila

■ GET

- Se a fila estiver vazia ficará bloqueada até chegar mensagem. Remove a primeira mensagem que estiver na fila

■ POLL

- Verificar se existe mensagem em uma fila específica e remover a primeira. Neste caso, não fica bloqueado

■ NOTIFY

- Definir uma função que será invocada sempre que uma mensagem for inserida na fila

Quando utilizar comunicação persistente por mensagens?

- Quando as partes estiverem interconectadas por meio de uma WAN, cuja probabilidade de desconexão temporária não for desprezível
- Quando o sistema precisar de comunicação assíncrona e persistente
- Quando o sistema precisar de desacoplamento temporal

Comunicação orientada a mensagens

Casos de uso

- **Integração de sistemas heterogêneos**
 - Diferentes linguagens, plataformas ou localizações geográficas
- **E-commerce e sistemas transacionais**
 - Carrinho, pagamento e envio processados por serviços desacoplados
- **Processamento assíncrono**
 - Geração de relatórios, uploads, tarefas em segundo plano
- **Sistemas IoT e Telemetria**
 - Dispositivos enviam dados para servidores via mensagens
- **Pipelines de dados**
 - ETL, análise em tempo real, integração com Kafka, Flink, etc.

Advanced Message Queuing Protocol (AMQP) I

Protocolo Avançado de Enfileiramento de Mensagens

- Padrão aberto OASIS para *middleware* orientado a mensagens
- Protocolo binário na camada de aplicação
- Requer um transporte confiável como o TCP
- Oferece suporte a segurança com autenticação e criptografia

Advanced Message Queuing Protocol (AMQP) II

Protocolo Avançado de Enfileiramento de Mensagens

- Suporte aos seguintes padrões de comunicação
 - **Ponto a ponto** – comunicação entre um emissor e um receptor
 - **Publicador-assinante** – comunicação entre um emissor e múltiplos receptores (*publish/subscribe*)
 - **Fila de mensagens** – comunicação entre um emissor e um receptor, qual pode ser intermediada por um ou mais servidores
- AMPQ oferece três níveis de entrega de mensagens
 - *at-most-once*
 - *at-least-once*
 - *exactly-once*

Advanced Message Queuing Protocol (AMQP) III

Protocolo Avançado de Enfileiramento de Mensagens

- Principais implementações
 - RabbitMQ – <https://www.rabbitmq.com>
 - Apache Qpid – <https://qpid.apache.org>
 - Apache ActiveMQ – <https://activemq.apache.org>

Casos de uso práticos do AMQP

■ Indústria e automação (SCADA)

- Sensores enviam dados e alarmes para sistemas de controle de forma confiável

■ Aviação e sistemas de reservas

- Companhias aéreas trocam informações sobre voos, reservas e check-in com aeroportos

■ Aplicações *web* e móveis

- Envio de mensagens entre usuários, notificações e atualizações de aplicativos

■ Monitoramento e operações (DevOps)

- Sistemas enviam *logs* e métricas por filas para ferramentas de análise e alertas

Message Queue Telemetry Transport (MQTT)

Protocolo de mensagens leve

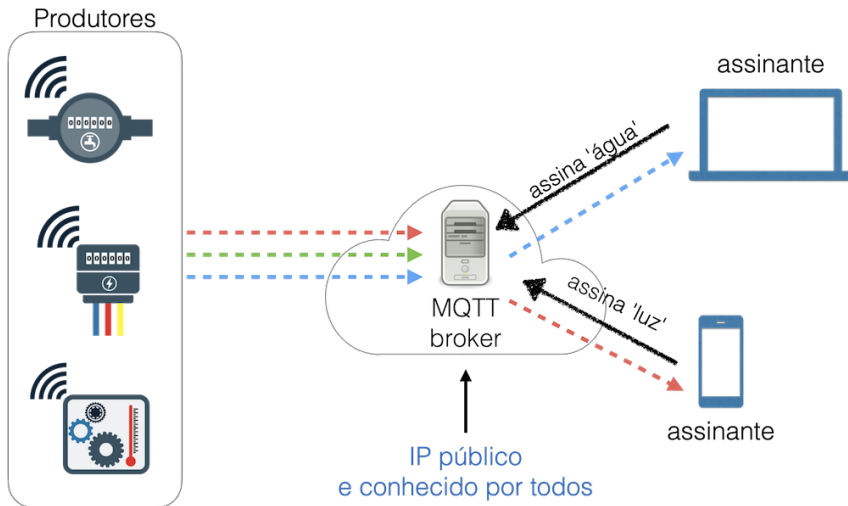
- Protocolo leve de mensageria baseado em **publish/subscribe**
- Ideal para dispositivos com recursos limitados e redes instáveis
- Funciona sobre TCP/IP e pode usar TLS para segurança
- Projetado para conexões de longa duração e baixa latência

Message Queue Telemetry Transport (MQTT)

Garantias de entrega

- **Níveis de QoS (*Quality of Service*)**
 - **QoS 0 (At-most-once)**
 - melhor desempenho, sem garantias
 - **QoS 1 (At-least-once)**
 - garante entrega, mas pode haver duplicatas
 - **QoS 2 (Exactly-once)**
 - garante entrega única (custo mais alto)
- **Segurança**
 - suporte a autenticação do cliente e criptografia de mensagens com TLS

Exemplo: cenário IoT e *broker* MQTT



Casos de uso práticos do MQTT

■ Internet das Coisas (IoT)

- Dispositivos como sensores, atuadores e eletrodomésticos trocam dados com *gateways* ou servidores na nuvem

■ Automação residencial

- Lâmpadas, termostatos e fechaduras inteligentes se comunicam por mensagens leves e rápidas

■ Monitoramento remoto

- Equipamentos em locais isolados (ex: tratores, boias marítimas, usinas) enviam dados com pouca largura de banda

■ Veículos conectados

- Envio de informações sobre localização, status e telemetria em tempo real

AMQP vs MQTT

Aspecto	AMQP	MQTT
Modelo de comunicação	<i>Queue/Exchange</i>	<i>Publish/Subscribe</i>
Peso do protocolo	Médio a alto	Leve
QoS	3 (at-most, at-least, exactly)	3 (0, 1, 2)
Segurança	SASL, TLS	TLS
Persistência de mensagens	Sim	Sim
Destinado à aplicações	Corporativas	IoT, sensores, <i>mobile</i>
Requer rede confiável	Sim	Não

Quando usar AMQP ou MQTT?

■ Use AMQP quando

- Precisa de roteamento complexo de mensagens
- Requer transações, filas duráveis e controle rigoroso de fluxo
- Atua em ambientes corporativos com serviços robustos

■ Use MQTT quando

- Está lidando com dispositivos embarcados ou IoT
- Precisa de um protocolo leve e tolerante a falhas de rede
- Deseja simplicidade e comunicação em tempo real (telemetria)

Aulas baseadas em



TANENBAUM, Andrew S; STEEN, Maarten van. **Sistemas Distribuidos: Princípios e paradigmas**. 2. ed.: Prentice Hall, 2008.